

10.8 Example: Breaking a Line into ``Words''

In an earlier assignment, an ``extra credit" version of a problem asked you to write a little checkbook balancing program that accepted a series of lines of the form

```
deposit 1000
check 10
check 12.34
deposit 50
check 20
```

It was a surprising nuisance to do this in an *ad hoc* way, using only the tools we had at the time. It was easy to read each line, but it was cumbersome to break it up into the word (``deposit" or ``check") and the amount.

I find it very convenient to use a more general approach: first, break lines like these into a series of whitespace-separated words, then deal with each word separately. To do this, we will use an *array of pointers to char*, which we can also think of as an ``array of strings," since a string is an array of *char*, and a pointer-to-*char* can easily point at a string. Here is the declaration of such an array:

```
char *words[10];
```

This is the first complicated C declaration we've seen: it says that `words` is an array of 10 pointers to *char*. We're going to write a function, `getwords`, which we can call like this:

```
int nwords;
nwords = getwords(line, words, 10);
```

where `line` is the line we're breaking into words, `words` is the array to be filled in with the (pointers to the) words, and `nwords` (the return value from `getwords`) is the number of words which the function finds. (As with `getline`, we tell the function the size of the array so that if the line should happen to contain more words than that, it won't overflow the array).

Here is the definition of the `getwords` function. It finds the beginning of each word, places a pointer to it in the array, finds the end of that word (which is signified by at least one whitespace character) and terminates the word by placing a `'\0'` character after it. (The `'\0'` character will overwrite the first whitespace character following the word.) Note that the original input string is therefore modified by `getwords`: if you were to try to print the input line after calling `getwords`, it would appear to contain only its first word (because of the first inserted `'\0'`).

```
#include <stddef.h>
#include <ctype.h>

getwords(char *line, char *words[], int maxwords)
{
    char *p = line;
    int nwords = 0;

    while(1)
    {
        while(isspace(*p))
            p++;

        if(*p == '\0')
            return nwords;

        words[nwords++] = p;
    }
}
```

```

while(!isspace(*p) && *p != '\0')
    p++;

if(*p == '\0')
    return nwords;

*p++ = '\0';

if(nwords >= maxwords)
    return nwords;
}

```

Each time through the outer `while` loop, the function tries to find another word. First it skips over whitespace (which might be leading spaces on the line, or the space(s) separating this word from the previous one). The `isspace` function is new: it's in the standard library, declared in the header file `<ctype.h>`, and it returns nonzero (``true") if the character you hand it is a space character (a space or a tab, or any other whitespace character there might happen to be).

When the function finds a non-whitespace character, it has found the beginning of another word, so it places the pointer to that character in the next cell of the `words` array. Then it steps through the word, looking at non-whitespace characters, until it finds another whitespace character, or the `\0` at the end of the line. If it finds the `\0`, it's done with the entire line; otherwise, it changes the whitespace character to a `\0`, to terminate the word it's just found, and continues. (If it's found as many words as will fit in the `words` array, it returns prematurely.)

Each time it finds a word, the function increments the number of words (`nwords`) it has found. Since arrays in C start at `[0]`, the number of words the function has found so far is also the index of the cell in the `words` array where the next word should be stored. The function actually assigns the next word and increments `nwords` in one expression:

```
words[nwords++] = p;
```

You should convince yourself that this arrangement works, and that (in this case) the preincrement form

```
words[++nwords] = p;           /* WRONG */
```

would *not* behave as desired.

When the function is done (when it finds the `\0` terminating the input line, or when it runs out of cells in the `words` array) it returns the number of words it has found.

Here is a complete example of calling `getwords`:

```

char line[] = "this is a test";
int i;

nwords = getwords(line, words, 10);
for(i = 0; i < nwords; i++)
    printf("%s\n", words[i]);

```

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)